Hands-on `benchopt`
Imaging benchmarks tutorial

Benchopt contributors

Sorry I failed to release for the tutorial....

Sorry I failed to release for the tutorial....

But you will get to ride on the edge!

```
pip install -U git+https://github.com/benchopt/benchopt
```

You will just need an env with deep-inverse, benchopt and optinonally ptwt for the wavelet baseline.

Running a benchmark!

Install and run the benchmark

Repository used in the tutorial:

`https://github.com/tomMoral/26-05_benchopt_tuto_icms`

Minimal workflow:

```
git clone https://github.com/tomMoral/26-05_benchopt_tuto_icms tuto_b
cd tuto_benchopt/01-denoising
pip install benchopt
benchopt install .
benchopt run .
```

⇒ Once `Benchopt` is installed, two commands are enough: install the benchmark, then run it.

What happens during install and run?

For this benchmark, `Benchopt` will automatically:

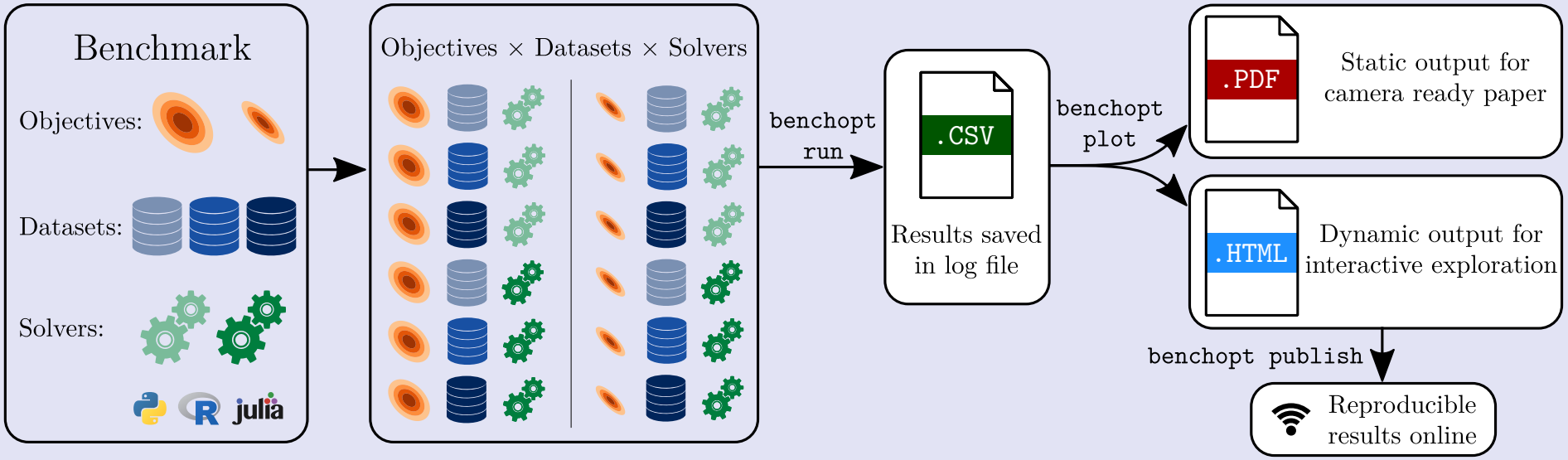
- Load the objective, dataset and available solvers.
- Install the benchmark requirements declared in the files.
- Run the available solver on the `CBSD` dataset.
- Collect metrics such as `psnr` and `ssim`.

In this repo:

- Dataset: `datasets/cbsd.py` — CBSD images, parametrized by σ
- Objective: `objective.py` — adds noise, computes PSNR
- Solvers: `solvers/wavelet.py`, `solvers/dncnn.py`
- Plot: `plots/denoised_images.py` – display results

Components of a benchmark

3 components: Objective, Dataset, Solver



Adding a denoiser solver!

Benchmark: `denoising`

Structure of a benchmark

```
benchmark/
├── objective.py
├── datasets/
│   ├── dataset1.py
│   └── dataset2.py
└── solvers/
    ├── solver1.py
    └── solver2.py
```

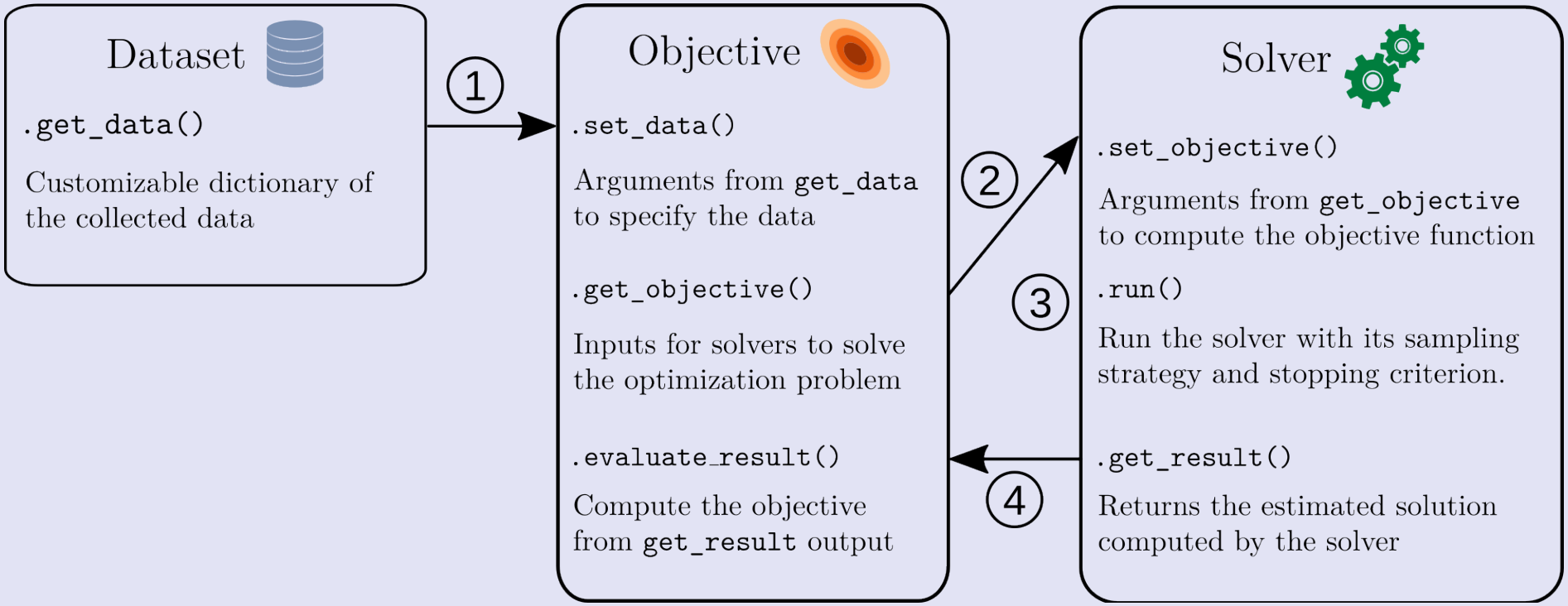
Modular & extendable

New solver? add a file
New dataset? add a file
New metric? modify objective

A modular framework to create benchmarks

3 components: Objective, Dataset, Solver

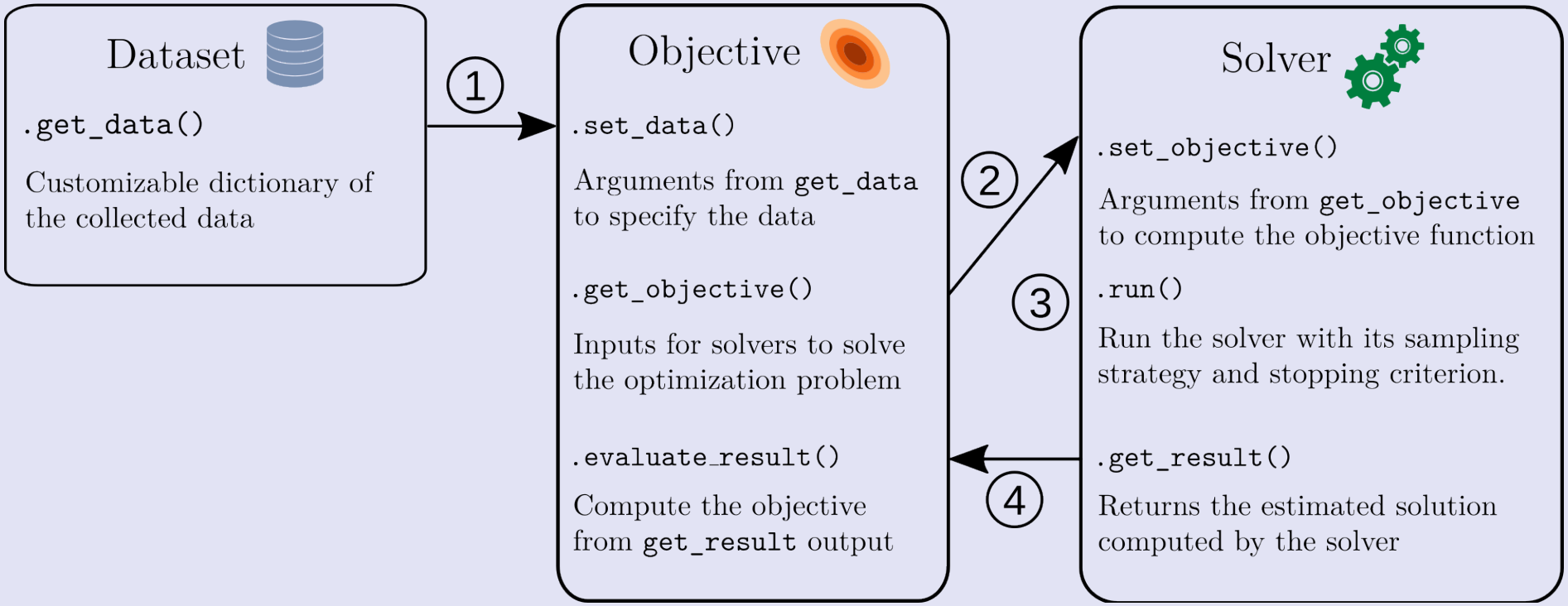
Dependency relation between Dataset - Objective - Solver



A modular framework to create benchmarks

3 components: Objective, Dataset, Solver

Dependency relation between Dataset - Objective - Solver



$\Rightarrow$ Benchopt defines the interface between components.

Explicit preprocessing

```python
from benchopt import BaseDataset

import deepinv as dinv


class Dataset(BaseDataset):

    name = "CBSD"
    parameters = {'sigma': [0.05, 0.1, 0.2]}

    def get_data(self):
        x_true = dinv.utils.load_example(
            'CBSD_0010.png', grayscale=False,
        )
        return dict(x_true=x_true, sigma=self.sigma)
```

Explicit preprocessing

```
from benchopt import BaseDataset

import deepinv as dinv


class Dataset(BaseDataset):

    name = "CBSD"
    parameters = {'sigma': [0.05, 0.1, 0.2]}

    def get_data(self):
        x_true = dinv.utils.load_example(
            'CBSD_0010.png', grayscale=False,
        )
        return dict(x_true=x_true, sigma=self.sigma)
```

$\Rightarrow$ Reproducible benchmark by design!

Goal: if you use the same setup, you don't need to re-run the baseline

Explicit evaluation protocol and requirements

```python
from benchopt import BaseObjective

import torch
import deepinv as dinv


class Objective(BaseObjective):

    name = "Denoising"
    requirements = ["pip::deepinv"]
    sampling_strategy = 'run_once'

    def set_data(self, x_true, sigma):
        self.x_true, self.sigma = x_true, sigma
        self.y = x_true + sigma * torch.randn(x_true.shape)

    def get_objective(self):
        return dict(y=self.y, sigma=self.sigma)

    def evaluate_result(self, x_hat):
        psnr = dinv.metric.PSNR()(x_hat, self.x_true).item()
        return dict(psnr=psnr)
```

Explicit parameters

```python
from benchopt import BaseSolver

import torch
import deepinv as dinv


class Solver(BaseSolver):

    name = "DnCNN"

    def set_objective(self, y, sigma):
        self.y, self.sigma = y, sigma

    def run(self, _):
        denoiser = dinv.models.DnCNN(
            pretrained='download', device=self.y.device
        )
        with torch.no_grad():
            self.x_hat = denoiser(self.y, self.sigma)

    def get_result(self):
        return dict(x_hat=self.x_hat)
```

Step 1: add a new solver file

Goal: add a new deep denoiser without changing the benchmark protocol.

- Open `solvers/skeleton.py` — your starting point
- Rename the solver (`name` attribute) to `DnCNN`
- Instantiate your denoiser in `run()`, e.g.:

```
dinv.models.DnCNN(pretrained='download', device=self.device)
```

- Keep `sampling_strategy = 'run_once'` — no loop needed
- Save the output in `self.x_hat`

$\Rightarrow$ Same interface, same objective, same dataset: only the model changes.

Step 2: plug in your denoiser

All deepinv denoisers share the same calling convention:

```
x_hat = denoiser(y, sigma)
```

Some models to try (all available with pretrained weights):

Model	Type	Key property
`DnCNN`	deep, fixed σ	fast, good baseline
`SCUNet`	deep, adaptive σ	robust across levels
`SwinIR`	transformer	state-of-the-art

$\Rightarrow$ Explore deepinv denoiser tour for the full list.

Step 3: validate and compare

Benchopt automatically discovers the new solver and compares it.

```
benchopt test . -k mysolver --skip-env
benchopt run . -s MySolver -s DRUNet -s WaveletDict
```

What to check:

- The solver file is picked up automatically.
- `get_result()` returns `{"x_hat": ...}`.
- `psnr` is reported alongside the existing solvers.
- No change needed in the objective or dataset.

$\Rightarrow$ Adding a solver means writing one compatible file — nothing else changes.

Writing the objective for blind deblurring!

Benchmark: `blind_deblur`

Step 1: understand what is given

Goal: implement `objective.py` for a blind deblurring benchmark. The solvers already exist — only the objective is missing.

Given for free (non-obvious, provided in the skeleton):

- `set_data()` — stores x, y, physics, `kernel_size`
- `get_one_result()` — delta kernel baseline

Yours to implement:

- `get_objective()` — what to expose to solvers
- `evaluate_result()` — dual metric (PSNR + kernel MSE)
- `save_final_results()` — persist both `x_hat` and `k_hat`

$\Rightarrow$ The `while callback()` pattern is illustrated by `dip_selfdeblur.py`.

Step 2: implement the objective methods

`get_objective`: expose `y` and `kernel_size` — do NOT expose the true kernel.

`evaluate_result`: dual metric

- Image quality: $\mathrm{PSNR}(\hat{x}, x_{\mathrm{true}})$
- Kernel accuracy: normalise both kernels, then compute MSE

$$\mathrm{kernel_mse} = \left\| \frac{\hat{k}}{\|\hat{k}\|_1} - \frac{k}{\|k\|_1} \right\|_2^2$$

`save_final_results`: persist both estimates to CPU

```
return dict(x_hat=x_hat.cpu(), k_hat=k_hat.cpu())
```

$\Rightarrow$ The stopping criterion monitors `psnr` — make sure it is in the returned dict.

Step 3: validate the objective

`Benchopt` runs both solvers and checks your objective.

```
benchopt test . --skip-env
benchopt run . -s Bilevel-PGD -s DIP-SelfDeblur
```

What to check:

- `psnr` and `kernel_mse` are reported for both solvers.
- The convergence curve improves over the delta-kernel baseline.
- The **Kernel** plot shows the estimated blur kernel per solver.
- Both solvers complete within the time budget.

$\Rightarrow$ Writing an objective means defining the protocol once — both solvers run against it automatically.

`Benchopt` Roadmap

- **Simpler solver evaluation** – make custom solver integration more direct, for instance with `eval_solver(run_func)`
- **Better visualization tools** – explore results more easily, in particular for image tasks
- **More reference benchmarks** – grow the benchmark collection with community contributions

$\Rightarrow$ `Benchopt` is by no means finished or complete!

It is useful to a growing group of people but we need to gather more feedbacks to fit the needs of the community.

Benchopt links and resources

- Documentation:
  benchopt.github.io
- Main repository:
  github.com/benchopt/benchopt
- Benchmark templates:
  github.com/benchopt/template_benchmark
  github.com/benchopt/template_benchmark_ml

$\Rightarrow$ If this tutorial was useful, please star the `benchopt` repo and share it.